

Performance Testing Report

Project: Enterprise Quantum Computing Platform — QFT Bank **Student:** Syed Muhammad Tahoor Ali **Diploma:** EduQual Level 6, Diploma in Artificial Intelligence Operations (ANPP-OP) **Environment:** Kubernetes deployment (Minikube), single-node, measured end-to-end through the NodePort

1. Purpose

This report records measured performance of the platform's main operations, taken end-to-end from an external client through the full request path (NodePort → nginx → backend → simulator). Its aim is to characterise realistic latency for each class of operation and to explain the large differences between them, which are inherent to the work each performs. All timings are real measurements.

2. Method

Each operation was invoked three times through the deployed frontend NodePort, with total round-trip time measured by the client (`curl's time_total`). Measuring through the NodePort — rather than against the backend directly — captures the realistic path a user experiences, including the nginx reverse proxy and in-cluster networking.

Three representative operations were tested, chosen to span the platform's cost range: a small circuit execution (Module 1), a Grover search (Module 3), and a full portfolio optimization (Module 2).

3. Results

Operation	Run 1	Run 2	Run 3	Character
Circuit execution (2 qubits, 1024 shots)	10.9 ms	174.4 ms	276.3 ms	Fast, single circuit
Grover search (3 qubits, 1024 shots)	661.7 ms	195.0 ms	106.7 ms	Fast, small algorithm
Portfolio optimization (6 assets, 30 iterations)	6.72 s	9.82 s	6.81 s	Heavy, iterative

4. Analysis

Circuit execution is the lightest operation: a single circuit built and run once on the simulator. Latency ranged from about 11 ms to 276 ms. The variation is dominated by cold-versus-warm effects — the first invocation after a period of inactivity pays simulator and code-path warm-up costs that later calls avoid. The internal execution time reported by the backend for this circuit was around 12 ms; the remainder is request handling, proxying, and networking.

Grover's search is also fast, in the ~100–660 ms range. It builds and runs a somewhat larger circuit (3 qubits with amplification iterations) but is still a single, bounded computation. Again the first run is the slowest, consistent with warm-up rather than with the algorithm itself being variable.

Portfolio optimization is roughly 30–90× slower than the other two, at ~6.7–9.8 seconds. This is expected and inherent: unlike the other operations, it does not run one circuit — it runs a QAOA circuit *once per optimiser iteration*, up to the configured cap (30 iterations here, with 29 actually run in the sampled execution). Each iteration is a full simulator execution. The wall-clock cost is therefore the single-circuit cost multiplied by the iteration count, plus the classical optimiser's own overhead. The run-to-run variation (6.72 s to 9.82 s) reflects how many iterations COBYLA needs before converging, which is not fixed.

5. Interpretation for Capacity Planning

The results divide cleanly into two classes with different operational implications.

Interactive operations (circuit design, individual algorithm demonstrations) complete in well under a second in the warm state. These are suitable for synchronous request/response in the web interface, as implemented.

Batch-style operations (portfolio optimization, and by extension any COBYLA-driven algorithm such as VQE) take seconds because they are iterative optimisations. In this proof-of-concept they run synchronously, which is acceptable for single-user demonstration. In a production, multi-user deployment these would be better handled asynchronously — submitted as jobs and polled for completion — so that a long optimisation does not hold an HTTP connection open. This is a natural evolution of the architecture rather than a defect; the API's execution-id model already returns an identifier per run, which is the foundation an asynchronous job pattern would build on.

6. Resource Context

These timings were taken on a single-node Minikube cluster sharing a laptop with the browser and tooling used for the measurement

itself. On dedicated or multi-node infrastructure, absolute latencies would improve and, more importantly, concurrent throughput would increase — the current single node serialises heavy simulator work. The relative pattern, however, is intrinsic: iterative quantum-classical optimisation will always cost substantially more than a single circuit execution, because it is many circuit executions by construction.

7. Conclusion

The platform's performance is well-characterised and matches expectations from first principles. Interactive quantum operations are sub-second when warm; iterative optimisation takes seconds in proportion to its iteration count. The measurements were taken end-to-end through the real deployment path, so they reflect what a user actually experiences. The clear separation into interactive and batch classes gives a concrete basis for how the platform would scale to production: keep interactive operations synchronous, and move iterative optimisations to an asynchronous job model.